Plume Network

Security Review

Cantina Managed review by:
Jiri123, Security Researcher

April 10, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Medium Risk . . . 4
3.1.1 `VaultComposerAsync` controller pending pollution breaks `updateRedeem` . . . 4
3.1.2 Asset-based finishRedeem creates hidden share deficits that let later fulfills drain older claims . . . 4
3.2 Low Risk . . . 5
3.2.1 Public Executor Can Force Refund And Consume Async Compose Message . . . 5
3.2.2 Inherited Public Composer Overloads Expose Permissioned OFT Bridges To Arbitrary Callers . . . 5
3.2.3 Async Cross-Chain Finish Redeems Revert On OFT Dust Rounding . . . 5
3.3 Informational . . . 6
3.3.1 Stale `oftCmd` Is Forwarded During Final Asset Send . . . 6
3.3.2 Async redeem escrow lets blocked holders cash out . . . 6
3.3.3 Missing `Permit2` Constructor Validation Can Brick Gasless Redeems . . . 6

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Plume is a public, EVM-compatible blockchain built for the next evolution of Real World Assets (RWAs).

From Feb 24th to Mar 15th the Cantina team conducted a review of nest-contracts on commit hash a754c310. The team identified a total of **8** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	3	2	1
Total	**8**	**7**	**1**

2.1 Scope

The security review had the following components in scope for nest-contracts on commit hash a754c310:

```
contracts
├── NestAccountant.sol
├── NestShareOFT.sol
├── NestShareSeizer.sol
├── NestVault.sol
├── NestVaultCore.sol
├── NestVaultOFT.sol
├── hooks
│   └── BlacklistHook.sol
├── libraries
│   ├── nest-vault
│   │   ├── NestVaultAccountingLogic.sol
│   │   ├── NestVaultAdminLogic.sol
│   │   ├── NestVaultCoreTypes.sol
│   │   ├── NestVaultCoreValidationLogic.sol
│   │   ├── NestVaultDepositLogic.sol
│   │   ├── NestVaultOperatorLogic.sol
│   │   ├── NestVaultRedeemLogic.sol
│   │   └── NestVaultTransferLogic.sol
│   └── vendor
│       └── cctp
│           ├── AddressUtils.sol
│           ├── BurnMessage.sol
│           ├── BurnMessageV2.sol
│           ├── MessageV2.sol
│           └── TypedMemView.sol
├── operators
│   └── NestVaultRedeemOperator.sol
└── ovault
    └── NestVaultComposer.sol
```

3 Findings

3.1 Medium Risk

3.1.1 `VaultComposerAsync` controller pending pollution breaks `updateRedeem`

Severity: Medium Risk

Context: NestVaultRedeemLogic.sol#L94-L96, VaultComposerAsyncUpgradeable.sol#L321-L334

Description: `VaultComposerAsyncUpgradeable._updateRequestRedeemAndSend()` assumes the vault bucket `pendingRedeem[controller = address(this)]` is only modified by composer bookkeeping (`totalPendingSharesSum`).

That invariant is not enforced in `NestVaultCore.requestRedeem()`: an authorized caller can choose any `_controller`, including the composer, as long as `_owner` authorization passes.

An attacker can directly call `requestRedeem(..., controller=composer, owner=attacker)` and inject shares into the composer controller bucket. Later, when a legitimate user sends `UpdateRedeemRequest` via compose, the composer expects `vault.updateRedeem` to return exactly the user delta, but vault returns (user delta + injected shares). This triggers `TransferInsufficient`, compose falls into refund handling, and the update payload is consumed without progress.

```
uint256 _returnAmount = _oldShares - _newShares;
IERC20 shareErc20 = IERC20(SHARE_ERC20());

uint256 preShareBalance = shareErc20.balanceOf(address(this));
INestVaultCore(address(VAULT())).updateRedeem($.totalPendingSharesSum, address(this),
↪  address(this));
uint256 postShareBalance = shareErc20.balanceOf(address(this));

if (_returnAmount != postShareBalance - preShareBalance) { // @audit
    revert Errors.TransferInsufficient();
}
```

User pending shares remain unchanged and repeated updates keep refunding while pollution remains.

Recommendation: Reject `requestRedeem(controller=composer)` unless `msg.sender == composer`.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.1.2 Asset-based finishRedeem creates hidden share deficits that let later fulfills drain older claims

Severity: Medium Risk

Context: VaultComposerAsyncUpgradeable.sol#L369-L386

Description: `VaultComposerAsyncUpgradeable._finishRedeemAndSend()` calculates a user's payout from that user's local `claimableRedeem.assets / claimableRedeem.shares` ratio, but it settles the payout by calling `VAULT().withdraw(assetAmount, address(this), address(this))`. In `NestVaultCore`, `withdraw()` does not consume the same `_shareAmount`; it burns controller-level claimable shares at the pooled `claimableRedeem[controller].shares / claimableRedeem[controller].assets` ratio.

When users have been fulfilled at different rates, those two ratios diverge. Finishing a high-ratio claim can therefore remove `_shareAmount` from the user's composer-side record while burning materially more than `_shareAmount` from the core controller's pooled claimable shares. Other users keep their original composer-side `claimableRedeem[*].shares`, but the core controller is left with fewer backing shares than those live claims assume.

That share shortfall stays hidden until new claimable shares appear. `_fulfillRedeem()` treats the fresh core shares as unaccounted claimable inventory and values them against the distorted pooled state, so a later fulfill can credit more assets than that fulfill actually realized.

Recommendation: Do not settle `finishRedeem` by withdrawing an asset amount from the pooled controller bucket. Settle by share count so the composer and core vault consume the same number of claimable shares.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Public Executor Can Force Refund And Consume Async Compose Message

Severity: Low Risk

Context: VaultComposerAsyncUpgradeable.sol#L172-L185

Description: `VaultComposerAsyncUpgradeable.lzCompose` catches `InsufficientMsgValue` and, when `_canRefund()` is true, immediately executes `_refund()` instead of reverting.

Because LayerZero endpoint `MessagingComposer.lzCompose()` is publicly callable by any executor, a third party can intentionally call compose with `msg.value < minMsgValue` (but high enough for refund path), forcing refund behavior and consuming the compose payload.

Recommendation: On `InsufficientMsgValue`, always revert so the payload remains retryable.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.2.2 Inherited Public Composer Overloads Expose Permissioned OFT Bridges To Arbitrary Callers

Severity: Low Risk

Context: VaultComposerSyncUpgradeable.sol#L210-L220

Description: `NestVaultComposer` adds `requiresAuth` to bridge entrypoints, but it still inherits public `depositAndSend` and `redeemAndSend` overloads from `VaultComposerSyncUpgradeable`.

Those inherited overloads still reach `_sendRemote()`. If the configured OFT is permissioned, such as `NestCCTPRelayer` or `NestVaultOFT`, any external caller can use the composer's authorization to exercise that bridge indirectly.

Recommendation: Add `requiresAuth` to `depositAndSend` and `redeemAndSend` overloads from `VaultComposerSyncUpgradeable`.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.2.3 Async Cross-Chain Finish Redeems Revert On OFT Dust Rounding

Severity: Low Risk

Context: VaultComposerAsyncUpgradeable.sol#L337-L339

Description: `VaultComposerAsyncUpgradeable._finishRedeemAndSend()` checks the locally withdrawn asset amount, then overwrites both `SendParam.amountLD` and `SendParam.minAmountLD` with `assetAmountReceived` before forwarding the transfer to the asset OFT.

```
// Update tracked vault claimable shares consumed from vault
$.totalFulfilledSharesSum =
    $.totalFulfilledSharesSum > sharesConsumed ? $.totalFulfilledSharesSum -
    ↪  sharesConsumed : 0;

emit Redeemed(_redeemer, _sendParam.to, _sendParam.dstEid, _shareAmount,
↪  assetAmountReceived);

// Update sendParam with the actual asset amount for the cross-chain send
_sendParam.amountLD = assetAmountReceived;
_sendParam.minAmountLD = assetAmountReceived;
```

```
_send(ASSET_OFT(), _sendParam, _refundAddress, _msgValue);
```

That is unsafe when the asset OFT uses the default `sharedDecimals() == 6` but the underlying asset has more than 6 local decimals. In that configuration, the OFT rounds the local amount down to the nearest transferable dust unit during debit. If the locally redeemed amount is not already aligned to that unit, the OFT's debited amount is lower than the composer-forced `minAmountLD`, so the send reverts.

For example, with an 8-decimal asset OFT the dust unit is `100`. If async finish withdraws `101`, the composer sends `amountLD = minAmountLD = 101`, but the OFT can only debit `100`, so the transfer fails on slippage.

As a result, async cross-chain redemption completion can fail for otherwise valid payouts on non 6 decimal asset OFTs whenever the redeemed amount is not dust-aligned.

Recommendation: Do not set the outbound asset OFT minimum to the raw local withdrawal amount. Instead, quantize the async finish payout to the OFT's transferable dust unit or mirror the sync path and avoid forcing `minAmountLD` to a value the OFT will deterministically round down.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 Stale `oftCmd` Is Forwarded During Final Asset Send

Severity: Informational

Context: VaultComposerAsyncUpgradeable.sol#L403-L407

Description: The function does not clear `_sendParam.oftCmd`, even though that field was previously used only to encode the redeem mode for composer-side control flow.

As a result, stale command data may be forwarded to the asset OFT send path. If the downstream OFT implementation interprets a non-empty `oftCmd` specially, this can cause unexpected behavior.

Recommendation: Clear `_sendParam.oftCmd` before calling `_send()` once the redeem type has already been decoded and consumed.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.

3.3.2 Async redeem escrow lets blocked holders cash out

Severity: Informational

Context: NestVaultRedeemLogic.sol#L120-L137

Description: `NestVault`'s async redeem lifecycle only enforces the share transfer hook at the initial `requestRedeem()` step. After shares enter escrow, every later state transition operates on vault-held inventory instead of user-held shares. `fulfillRedeem()` burns shares from `address(this)`, and `withdraw()`, `redeem()` pay assets out of the vault's pooled claimable bucket. Because `NestShareOFT` only authenticates `from` inside `_update()`, any later blacklist, lock, sender-based restriction on the original holder is irrelevant once the shares are sitting inside the vault.

Recommendation: Rebind the original holder identity throughout the async lifecycle instead of only at `requestRedeem()`.

Plume Network: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Missing `Permit2` Constructor Validation Can Brick Gasless Redeems

Severity: Informational

Context: NestVaultPermit2.sol#L27-L29

Description: `NestVaultPermit2` stores the Permit2 dependency as an immutable constructor argument but never validates it against `address(0)`.

Unlike other critical addresses in the vault stack, that constructor input is never validated against `address(0)`.

If `_permit2` is accidentally deployed as the zero address, both Permit2-assisted user entrypoints become permanently unusable for that implementation:

Recommendation: Reject `address(0)` in the `NestVaultPermit2` constructor.

Plume Network: Fixed in PR 83.

Cantina Managed: Fix verified.